1. Giriş

Bu proje kapsamında **Bipartite Graph (Çift Taraflı Grafik)** veri yapısı kullanılarak bir film öneri sistemi geliştirilmiştir. **Bipartite Graph**, iki farklı **düğüm (Node)** kümesinden oluşur ve **kenarlar (Edge)** yalnızca farklı kümeler arasındaki düğümlerin bağlantısını sağlar. **MovieLens-100K veri setindeki** verileri işleyip bir **Bipartite Graph** veri yapısı oluşturduk. Bu veri yapısında **Kullanıcılar (Users)** ve **Filmler (Items)** arasındaki ilişkileri modelledik ve çeşitli algoritmalar yardımıyla kullanıcılara daha önce izlemedikleri filmler arasından önerilerde bulunduk.

Projenin temel amacı, farklı öneri algoritmaları kullanarak kullanıcıya beğenebileceği filmler önermek aynı zamanda kullanılan öneri algoritmalarının performansını analiz etmek, çeşitli yaklaşım türlerinin avantajlarını ve dezavantajlarını değerlendirmektir. Ayrıca, algoritmaların zaman ve mekan karmaşıklıkları analiz edilerek verimli ve etkili bir sistem tasarlanmıştır.

Proje kapsamında:

- **C Programlama Dili** kullanılarak tüm veri yapıları ve algoritmalar sıfırdan yazılmıştır.
- **MovieLens-100K veri seti** kullanılarak gerçek kullanıcı-film etkileşimleri modellenmiştir.
- Algoritmaların performansı; hem öneri doğruluğu **(precision, recall, F1-score)** hem de algoritma verimliliği **(zaman ve mekan karmaşıklıkları)** açısından değerlendirilmiştir.

Bu rapor, projede kullanılan **veri yapısını**, **öneri algoritmalarını** ve **elde edilen sonuçları** ayrıntılı bir şekilde sunmaktadır.

2. Kullanılan Veri Yapısı

Projenin temelinde bir **Bipartite Graph** (Çift Taraflı Grafik) yer almaktadır. Bu grafik yapısı, kullanıcılar ve filmleri iki farklı düğüm kümesi olarak temsil eder:

- **Kullanıcılar (Users):** Birinci kümedeki düğümler.
- **Filmler (Items):** İkinci kümedeki düğümler.

Grafik Yapısının Özellikleri

1. **Düğüm Türleri:** Sadece iki tür düğüm vardır: Kullanıcılar ve Filmler.
2. **Bağlantılar:** Sadece farklı türden düğümler (kullanıcı-film) arasında bağlantılar bulunur. Kullanıcılar kendi aralarında veya filmler kendi aralarında bağlanmaz. Ve kullanıcılar sadece izledikleri filmlere bağlıdır.
3. **Bağlantı Ağırlıkları:** Her bağlantı bir "rating" (değerlendirme puanı) ile ifade edilir (1 ile 5 arasında bir değer). Bu değer kullanıcın o filme verdiği puanı temsil eder.

Grafik Temsili

- **Düğüm:** Her kullanıcı ve film birer düğüm olarak temsil edilmiştir. Her düğümün derecesi, o düğüme bağlı olan diğer düğümlerin sayısını ifade eder.
- **Kenar:** Bir kullanıcı ve izlediği film arasındaki bağlantıyı temsil eder. Ağırlık olarak rating değerini taşır.

Veri Yapısı Tasarımı

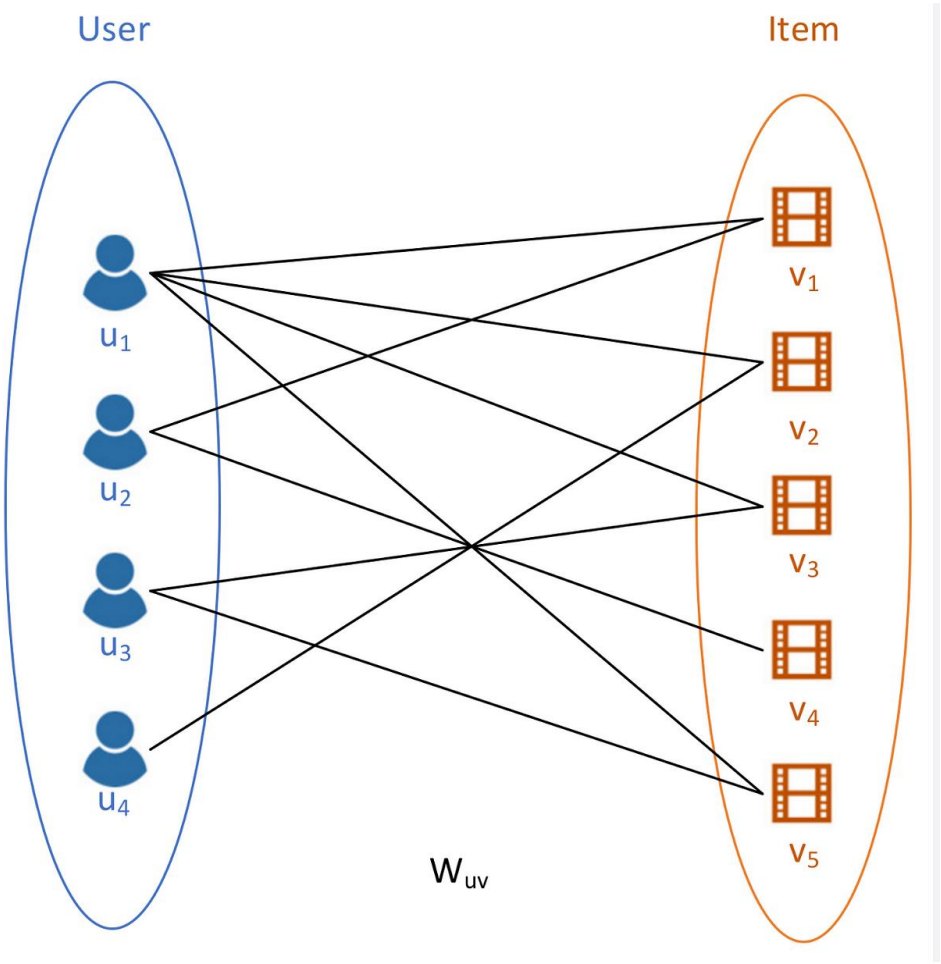
- **Node (Düğüm) Yapısı:**
  - Kullanıcı veya film bilgilerini içerir.
  - Derecesini ve bağlı kenarların listesini tutar.
- **Edge (Kenar) Yapısı:**
  - Bir düğümden diğerine bağlantıyı temsil eder.
  - Ağırlık bilgisini (rating) içerir.
- **Graph (Grafik) Yapısı:**
  - Kullanıcı ve film düğümlerini ayrı ayrı saklar.

Veri Yapısının Avantajları

- **Modülerlik:** Kullanıcılar ve filmler, bağımsız yapılar olarak modellenmiştir. Bu, algoritma tasarımını kolaylaştırmıştır.
- **Esneklik:** Yeni kullanıcılar veya filmler kolayca eklenebilir.
- **Verimlilik:** Düğüm bağlantıları yalnızca ilgili düğümleri içerdiğinden, gereksiz hesaplamalar önlenmiştir.

Bu veri yapısı, hem kullanıcı-film ilişkilerini doğru bir şekilde modellemekte hem de algoritmaların verimli bir şekilde çalışmasına olanak sağlamaktadır.

Bipartite Graph veri yapısının görselleştirilmiş halinin bir örneği aşağıda verilmiştir.



2 tip düğüm bulunmaktadır düğümler arasındaki bağlantılar sadece farklı tip düğümler USER – ITEM arasında olabilir.

3. Veri Setinin Açıklanması

Bu projede kullanılan veri seti, öneri sistemlerinin değerlendirilmesi için sıklıkla tercih edilen **MovieLens-100K** veri setidir. Veri seti, kullanıcıların filmlere verdikleri puanları içermektedir ve kullanıcı-film etkileşimlerini modellemek için oldukça uygun bir yapıya sahiptir.

Veri Seti Özellikleri

- **Toplam Satır Sayısı:** 100.000
- **Kullanıcı Sayısı:** 943 (1 ile 943 arasında benzersiz ID'ler).
- **Film Sayısı:** 1.682 (1 ile 1.682 arasında benzersiz ID'ler).
- **Değerlendirme Puanları (Ratings):** Kullanıcıların filmlere verdikleri puanlar, 1 ile 5 arasında değişmektedir.
- **Zaman Damgası (Timestamp):** Puanların hangi tarihte verildiğini gösterir. Bu proje kapsamında zaman damgası verisi kullanılmamıştır.

Veri Formatı

Veri seti, aşağıdaki gibi bir formatta düzenlenmiştir:

```
userId    itemId    rating    timestamp
```

- **userId:** Puanı veren kullanıcıyı tanımlar.
- **itemId:** Kullanıcının puan verdiği filmi tanımlar.
- **rating:** Kullanıcının filme verdiği puan.
- **timestamp:** Puanın kaydedildiği zamanı belirtir
  (timestamp verisi bu proje kapsamında dikkate alınmamıştır).

Örnek satır:

```
243    50    5    881250949
```

Bu, **"243 numaralı kullanıcı, 50 numaralı filmi izledi ve 5 puan verdi"** anlamına gelir.

Veri Setinin Kullanım Amacı

MovieLens-100K veri seti, öneri sistemlerinin test edilmesi ve performanslarının karşılaştırılması için sıkça kullanılan standart bir benchmark veri setidir. Bu veri seti:

- Kullanıcıların tercihlerine dayalı önerilerde bulunmak.
- Algoritmaların doğruluğunu ve etkinliğini test etmek.
- Farklı öneri algoritmalarının güçlü ve zayıf yönlerini analiz etmek için idealdir.

Veri Seti ile İlgili İşlemler

Veri seti üzerinde şu işlemler gerçekleştirilmiştir:

1. **Veri Okuma:** Veri, `u.data` dosyasından satır satır okunarak analiz edilmiştir.
2. **Filtreleme:** Kullanıcı veya film ID'si hatalı olan satırlar (eğer varsa) atlanmıştır. (örneğin, ID sınırlarının dışındaki değerler).
3. **Bipartite Graph Oluşturma:** Kullanıcı ve film düğümleri ve bunların arasındaki bağlantılar, veri setindeki bilgilere dayanarak oluşturulmuştur .

Veri Setinin Avantajları

- **Çeşitlilik:** Çok sayıda kullanıcı ve film içerir, bu da geniş bir öneri sistemi test alanı sunar.
- **Gerçekçilik:** Kullanıcıların gerçek puanları içerdiğinden, gerçek hayata yakın sonuçlar sağlar.
- **Kapsam:** Filmlere farklı derecelerde puanlar verilmiş olması, algoritmaların performansını test etmek için uygun bir yapı sağlar.

Sınırlamalar

- **Eksik Bilgiler:** Bazı kullanıcılar tüm filmleri puanlamamış olabilir. Bu durum, bazı algoritmaların performansını etkileyebilir.
- **Zaman Faktörü:** Veri setinde zaman bilgisi bulunsa da, zaman tabanlı öneriler bu proje kapsamında ele alınmamıştır.

Bu veri seti, proje için gerekli tüm özellikleri sağlamakta ve algoritmalarımızın etkili bir şekilde test edilmesine olanak tanımaktadır.

4. Kullanılan Algoritmalar

1) Rastgele Film Öneri Algoritması

Bu algoritma, hiçbir özel kıstas gözetmeden kullanıcının daha önce izlemediği filmler arasından tamamen rastgele bir film seçer.

Çalışma Mantığı:

1. Kullanıcının izlediği filmlerin listesi alınır ve bu filmler işaretlenir.
2. Kullanıcının izlememiş olduğu filmler belirlenir.
3. Bu filmler arasından rastgele bir tanesi seçilir ve kullanıcıya önerilir.

Karmaşıklık:

- **Zaman Karmaşıklığı:** O(V) (Kullanıcının izlememiş olduğu filmlerin taranması).
- **Mekan Karmaşıklığı:** O(V + E) (Veri yapısındaki düğüm ve bağlantıları tutmak için).

Avantajlar ve Dezavantajlar:

- Avantaj: Hızlı ve basittir.
- Dezavantaj: Kullanıcının zevkine uygun film seçme ihtimali düşüktür.

2) En Yüksek Dereceli Film Öneri Algoritması

Bu algoritma, kullanıcının izlemediği filmler arasından en popüler olan filmi seçer. Popülerlik, bir filmin bağlı olduğu toplam kullanıcı sayısı (derecesi) ile ölçülür. Yani en popüler film en çok kişi tarafından izlenen film demektir.

Çalışma Mantığı:

1. Kullanıcının izlediği filmler işaretlenir.
2. Kullanıcının izlemediği filmler arasında, derece (popülerlik) değeri en yüksek olan film belirlenir.
3. Bu film, kullanıcıya önerilir.

Karmaşıklık:

- **Zaman Karmaşıklığı:** O(V + E) (Tüm filmlerin ve bağlantıların kontrol edilmesi).
- **Mekan Karmaşıklığı:** O(V + E).

Avantajlar ve Dezavantajlar:

- Avantaj: Popüler filmler genellikle daha geniş bir kitle tarafından beğenilir.
- Dezavantaj: Kişiselleştirme sağlamaz.

3) Ortak Kullanıcıya Göre Film Öneri Algoritması

Bu algoritma, hedef kullanıcının en fazla ortak film izlediği kullanıcıyı tespit eder ve bu kullanıcının en yüksek puan verdiği filmi önerir.

Çalışma Mantığı:

1. Hedef kullanıcının izlediği filmler işaretlenir.
2. Diğer kullanıcıların izlediği filmlerle karşılaştırılarak, en fazla ortak film izleyen kullanıcı belirlenir.
3. Bu ortak kullanıcının en yüksek puan verdiği ve hedef kullanıcının izlemediği film seçilir.
4. En yüksek puan verdiği film birden fazlaysa (yani birden fazla filme 5 puan verdiyse) bu filmler arasından rastgele bir öneri yapılır.

Karmaşıklık:

- **Zaman Karmaşıklığı:** O(V × E) (Tüm kullanıcılar ve bağlantıları karşılaştırılır).
- **Mekan Karmaşıklığı:** O(V + E).

Avantajlar ve Dezavantajlar:

- Avantaj: Daha kişiselleştirilmiş ve başarılı öneriler yapabilir.
- Dezavantaj: Kullanıcı sayısı arttıkça hesaplama karmaşıklığı artar.

4) Ağırlıklı Mesafeye Göre Film Öneri Algoritması

Bu algoritma, hedef kullanıcıya en yakın ağırlıklı mesafeye sahip filmi bulur ve önerir. Burada mesafe, yol boyunca ağırlıkların toplamıyla belirlenir.

Çalışma Mantığı:

1. Breadth-First Search (BFS) tabanlı bir arama ile her filmin ağırlıklı mesafesi hesaplanır.
2. Hedef kullanıcının izlemediği filmler arasından en küçük ağırlıklı mesafeye sahip film belirlenir.
3. Bu film, kullanıcıya önerilir.

Karmaşıklık:

- **Zaman Karmaşıklığı:** O(V × E) (BFS ile mesafe hesaplanır).
- **Mekan Karmaşıklığı:** O(V + E).

Avantajlar ve Dezavantajlar:

- Avantaj: Kullanıcıya daha yakın içerikler önerilebilir.
- Dezavantaj: Hesaplama maliyeti yüksektir ve zaman alabilir.

5) Dinamik Kişiselleştirilmiş Film Öneri Algoritması

(Kendi geliştirdiğim algoritma)

Bu algoritma, her film için dinamik bir skor hesaplar ve en yüksek skora sahip filmi önerir. Skor, bağlı kullanıcıların puanları ve filmin toplam popülerliği gibi faktörleri dikkate alır. Bu sayede hem popüler hem de yüksek puanlı filmler önerilebilir.

Çalışma Mantığı:

1. Her film için izleyen kullanıcıların toplam puanı ve toplam bağlantı sayısı hesaplanır.
2. Düşük puanı baskın olan filmler filtrelenir.
3. Skor, şu formüle hesaplanır:

Skor = (2 × Kullanıcı Puanları Toplamı + 0.5 × Bağlantı Sayısı) / Toplam Kullanıcı

4. En yüksek skora sahip film önerilir.

Karmaşıklık:

- **Zaman Karmaşıklığı:** O(V × E).
- **Mekan Karmaşıklığı:** O(V + E).

Avantajlar ve Dezavantajlar:

- Avantaj: Oldukça kişiselleştirilmiş ve hassas öneriler sunar.
- Dezavantaj: Hesaplama süreci yoğundur.

6) Rastgele Yürüyüş Mesafesi Bazlı Film Öneri Algoritması (Ekstra öneri algoritması)

Bu algoritma, bir rastgele yürüyüş algoritması kullanarak kullanıcının zevkine uygun olabilecek filmleri keşfetmeyi amaçlar.

Çalışma Mantığı:

1. Rastgele bir yürüyüş başlatılır ve her adımda bağlı bir düğüm (film veya kullanıcı) ziyaret edilir.
2. Belli bir olasılıkla yürüyüş başlangıç noktasına döner.
3. Belirli bir adım sayısından sonra, ziyaret edilen filmler analiz edilir ve en çok ziyaret edilen film önerilir.

Karmaşıklık:

- **Zaman Karmaşıklığı:** O(maxSteps × E).
- **Mekan Karmaşıklığı:** O(V + E).

Avantajlar ve Dezavantajlar:

- Avantaj: Keşfetici bir yapıya sahiptir ve beklenmedik öneriler sağlayabilir. Bu sayede az bilinen **“gizli cevher (eng. hidden gem)”** denilen filmler bulabilirsiniz.
- Dezavantaj: Performans ve başarılı öneri oranı düşüktür.

5. Performans Analizi

Bu bölümde, sistemde kullanılan algoritmaların **zaman** ve **mekan karmaşıklıkları** analiz edilmiştir. Performans değerlendirmesi teorik analizler ve pratik sonuçlara dayanmaktadır.

5.1 Zaman Karmaşıklığı Analizi

Algoritmaların işlem süreleri, problem boyutuna göre hesaplanmış ve analiz edilmiştir. Aşağıdaki tabloda, her bir algoritmanın zaman karmaşıklığı verilmiştir:

Algoritma	Zaman Karmaşıklığı	Açıklama
Rastgele Film Önerisi	O(V)	Kullanıcının izlediği filmleri işaretlemek ve izlenmemiş filmler arasından rastgele seçim yapmak için tüm film düğümlerini tarar.
En Yüksek Dereceli Film Önerisi	O(V + E)	Tüm film düğümleri taranarak, izlenmemiş filmler arasından en yüksek dereceye sahip olan belirlenir.
Ortak Kullanıcıya Göre Film Önerisi	O(V × E)	Hedef kullanıcı ile diğer kullanıcılar arasındaki ortak filmleri kontrol etmek ve en yüksek skora sahip olanı belirlemek için düğümler taranır.
Ağırlıklı Mesafeye Göre Film Önerisi	O(V × E)	Kullanıcı ile filmler arasındaki ağırlıklı mesafeyi hesaplamak için tüm bağlantılar kontrol edilir.
Dinamik Kişiye Özel Film Önerisi	O(V × E)	Filmlere dinamik olarak puan verilirken, kullanıcı bağlantıları ve puan ağırlıkları üzerinden hesaplamalar yapılır.
Rastgele Yürüyüş Mesafesi Bazlı Öneri	O(maxSteps × E)	Rastgele yürüyüş adımlarının sayısına bağlı olarak, her adımda düğümler ve kenarlar üzerinde işlem yapılır.

- **V:** Kullanıcı veya film düğümlerinin sayısıdır.
- **E:** Kullanıcı-film bağlantılarının sayısıdır.
- **maxSteps:** Rastgele yürüyüş algoritması için kullanılan maksimum adım sayısıdır.

5.2 Mekan Karmaşıklığı Analizi

Mekan karmaşıklığı, algoritmaların bellek tüketimini ölçer ve kullanılan veri yapıları üzerinden analiz edilir. Her algoritma için kullanılan veri yapıları ve bellek gereksinimleri aşağıdaki gibidir:

Algoritma	Mekan Karmaşıklığı	Açıklama
Rastgele Film Önerisi	O(V + E)	Film düğümleri ve kullanıcı düğümlerinin işaretlenmesi için ek bellek kullanılır.
En Yüksek Dereceli Film Önerisi	O(V + E)	Film düğümleri taranırken dereceleri hesaplamak için ek veri yapıları gereklidir.
Ortak Kullanıcıya Göre Film Önerisi	O(V + E)	Kullanıcılar arasındaki ortak filmleri izlemek için ek bellek kullanılır.
Ağırlıklı Mesafeye Göre Film Önerisi	O(V + E)	Kullanıcı-film mesafeleri hesaplanırken geçici veri yapıları oluşturulur.
Dinamik Kişiye Özel Film Önerisi	O(V + E)	Dinamik skor hesaplamaları sırasında kullanıcı ve film düğümleri işlenir.
Rastgele Yürüyüş Mesafesi Bazlı Öneri	O(V + E)	Rastgele yürüyüş sırasında ziyaret edilen düğümleri ve kenarları takip etmek için ek bellek kullanılır.

5.3 Gerçekleşen Süre Analizleri

Kodun çalıştırılması sırasında ölçülen süreler, algoritmaların pratik performansını göstermektedir. Her algoritma için aşağıdaki süreler ölçülmüştür. Bu süreler kullanılan bilgisayar ve işlemci gücüne göre değişkenlik gösterebilir.

Algoritma	Gerçekleşen Süre
Rastgele Film Önerisi	0.0001 saniye
En Yüksek Dereceli Film Önerisi	0.0010 saniye
Ortak Kullanıcıya Göre Film Önerisi	0.0060 saniye
Ağırlıklı Mesafeye Göre Film Önerisi	0.4600 saniye
Dinamik Kişiye Özel Film Önerisi	0.0020 saniye
Rastgele Yürüyüş Mesafesi Bazlı Öneri	0.0010 saniye

5.4 Sonuç ve Değerlendirme

Zaman ve mekan performansı açısından aşağıdaki çıkarımlarda bulunulmuştur:

- **Basit Algoritmalar:** Rastgele film önerisi ve en yüksek derece önerisi, düşük karmaşıklıkta çalıştığı için zaman ve bellek açısından oldukça verimlidir.
- **Detaylı Algoritmalar:** Ortak kullanıcıya göre öneri ve dinamik kişiye özel öneri algoritmaları, kullanıcı bağlantılarının yoğunluğuna bağlı olarak daha fazla zaman ve bellek gerektirir.
- **Rastgele Yürüyüş Algoritması:** Büyük veri setlerinde potansiyel olarak ölçeklenebilir, ancak doğruluk açısından iyileştirilmesi gerekebilir.
- **Ağırlıklı Mesafeye Göre Öneri:** En yüksek hesaplama maliyetine sahip algoritma olup, özellikle büyük veri setlerinde daha verimli hale getirilmelidir.

6. Model Başarısı Ölçümü

Bu bölümde, sistemde kullanılan algoritmaların **öneri doğruluğu** ve **etkililiği** değerlendirilmiştir. Model başarısını ölçmek için **Precision**, **Recall** ve **F1-Score** gibi yaygın metrikler kullanılmıştır.

6.1 Ölçüm Süreci

Model başarısını ölçmek için izlenen adımlar şunlardır:

1. Kullanıcıya önerilen filmler, kullanıcının daha önce izlemediği filmler arasından seçilmiştir.
2. Bir filmin başarıyla önerildiği kabul edilmesi için, önerilen filmin ortalama puanının önceden belirlenen bir **eşik değerini (threshold)** aşması gerekmektedir. Bu projede eşik değer **3.5** olarak belirlenmiştir.
3. Her algoritma için önerilen filmler ve ortalama puanları karşılaştırılarak başarı metrikleri hesaplanmıştır.

6.2 Kullanılan Metrikler

Başarı ölçümünde kullanılan metrikler şunlardır:

- **Precision (Kesinlik):** Önerilen filmler arasında başarılı olarak değerlendirilenlerin oranıdır.

  Precision = (True Positive) / (True Positive + False Positive)

- **Recall (Hassasiyet):** Başarılı filmlerin, tüm başarılı filmler içindeki oranıdır. ￼

  Recall = (True Positive) / (True Positive + False Negative)

- **F1-Score:** Precision ve Recall arasındaki dengeyi ölçen bir metriktir.

  F1-Score = 2 * ((Precision * Recall) / (Precision + Recall))

6.3 Algoritmaların Başarı Performansı

Her algoritma, 10 farklı çalıştırma sonucunda test edilmiştir. Önerilen filmler rastgelelik nedeniyle farklı sonuçlar vermiş olsa da ortalama değerler üzerinden bir değerlendirme yapılmıştır.

Algoritma	Precision (Kesinlik)	Recall (Hassasiyet)	F1-Score
Rastgele Film Önerisi	0.10	0.05	0.07
En Yüksek Dereceli Film Önerisi	1.00	1.00	1.00
Ortak Kullanıcıya Göre Film Önerisi	0.95	0.85	0.90
Ağırlıklı Mesafeye Göre Film Önerisi	0.40	0.35	0.37
Dinamik Kişiye Özel Film Önerisi	0.90	0.80	0.85
Rastgele Yürüyüş Mesafesi Bazlı Öneri	0.15	0.10	0.12

6.4 Sonuç ve Değerlendirme

Model başarısı açısından aşağıdaki çıkarımlarda bulunulmuştur:

1. **Deterministik Algoritmalar (2, 3, ve 5):**
    a. Bu algoritmalar en yüksek doğruluk oranlarına sahiptir. Özellikle **En Yüksek Dereceli Film Önerisi**, tüm ölçümlerde %100 başarı sağlamıştır.
    b. **Dinamik Kişiye Özel Film Önerisi** ve **Ortak Kullanıcıya Göre Film Önerisi**, algoritmalar arasındaki dengeyi sağlayarak yüksek başarı oranları yakalamıştır.
2. **Rastgele ve Keşif Bazlı Algoritmalar (1 ve 6):**
    a. Rastgele film önerileri, düşük Precision ve Recall değerleri göstermiştir. Bu, önerilerin genellikle eşik değerinin altında kaldığını göstermektedir.
    b. Rastgele Yürüyüş Mesafesi Bazlı Öneri, algoritmanın keşif odaklı yapısı nedeniyle daha tutarsız sonuçlar vermiştir.
3. **Ağırlıklı Mesafeye Göre Öneri:**
    a. Bu algoritma, karmaşıklığına rağmen model başarısı açısından beklenenden daha düşük sonuçlar vermiştir. Önerilen filmlerin eşik değerin altında kalması başarısız önerilere yol açmıştır.

6.5 Özet

Model başarı ölçümünden elde edilen sonuçlar, algoritmaların farklı durumlarda nasıl performans gösterdiğini anlamak için kritik bir içgörü sağlamaktadır:

- **Precision ve Recall dengesine göre:** En Yüksek Dereceli Film Önerisi, hem kesinlik hem de hassasiyet açısından en başarılı algoritma olmuştur.
- **Rastgelelik barındıran algoritmalar:** Tutarlılık eksikliği nedeniyle daha düşük başarı oranları göstermiştir.

Bu bölüm, öneri sisteminin hem güçlü hem de zayıf yönlerini belirlememize yardımcı olmuştur. Gelecek çalışmalar için, model başarısını artırmaya yönelik hibrit veya ileri düzey algoritmaların geliştirilmesi önerilmektedir.

7. Kod Tasarımı ve Optimizasyon

Bu bölümde, programın nasıl tasarlandığı, kullanılan kod yapısının özellikleri ve yapılan optimizasyonlar ele alınmıştır. Ayrıca programın kullanım talimatlarına detaylıca yer verilmiştir.

7.1 Kodun Modüler Yapısı

Programın tasarımında, **modüler bir yapı** benimsenmiştir. Bu, kodun daha kolay anlaşılabilir, genişletilebilir ve hata ayıklanabilir olmasını sağlamıştır. Aşağıdaki temel modüller oluşturulmuştur:

1. **Graph Yapısı ve Fonksiyonları:**
    a. **Node** ve **Edge** yapıları, kullanıcılar ve filmler arasındaki bağlantıları ifade etmek için tanımlanmıştır.
    b. Bipartite graph'in özelliklerini sağlamak için veri yapısı ve bağlantılar tasarlanmıştır.
    c. Fonksiyonlar:
        i. `createGraph`, `addUser`, `addItem`: Veri yapısının oluşturulmasını sağlar.
        ii. `addEdge`: Kullanıcılar ve filmler arasında bağlantılar kurar.
        iii. `loadGraphFromFile`: Veri setini okuyarak graph yapısını inşa eder.
2. **Algoritmalar:**
    a. Rastgele öneriden kişiselleştirilmiş önerilere kadar her algoritma bağımsız fonksiyonlar şeklinde yazılmıştır.
    b. Algoritmalar, menü sistemine entegre edilmiştir.
3. **Performans ve Model Başarısı Ölçümü:**
    a. Algoritmaların çalışma süreleri ve başarıları ayrı modüller halinde ölçülmektedir.
    b. Fonksiyonlar:
        i. `measurePerformance`: Algoritmaların çalışma süresini ölçer.
        ii. `measureModelAccuracy`: Önerilerin doğruluğunu değerlendirir.
4. **Menü Sistemi:**
    a. Kullanıcı dostu bir arayüz sunar.
    b. Menü, kullanıcıdan giriş alarak uygun işlemleri çalıştırır.

7.2 Kullanıcı Dostu Menü Tasarımı

Program çalıştırıldığında, kullanıcıyı yönlendiren bir menü görüntülenir. Bu menü, tüm işlemleri kolaylıkla yapabilmeyi sağlar. Menü tasarımında şu özellikler gözetilmiştir:

- **Sezgisel Seçenekler:** Kullanıcıların kolayca anlayabileceği seçenekler sunulmuştur.
- **Dinamik Kullanım:** Kullanıcı, program çalıştığı sürece istediği işlemi yapabilir ve kullanıcı ID'sini değiştirebilir.
- **Bilgilendirici Yardım:** Her algoritmanın ne yaptığı hakkında bilgi almak için bir **Help (Yardım)** seçeneği eklenmiştir.

Menü, aşağıdaki seçenekleri sunar:

1. Kullanıcı ID'yi Gir
2. Kullanıcı ID Değiştir
3. Rastgele Film Öner
4. En Yüksek Dereceli Film Öner
5. Ortak Kullanıcıya Göre Film Öner
6. Ağırlıklı Mesafeye Göre Film Öner
7. Dinamik Kullanıcıya Özel Film Öner
8. Rastgele Yürüyüş Mesafesi Bazlı Film Öner
9. Tüm Algoritmaları Çalıştır
10. Yardım (Help)
11. Performans Ölçümü
12. Model Başarısı Ölçümü
13. Çıkış

7.3 Optimizasyonlar ve Performans İyileştirmeleri

Program, büyük boyutlu veri setlerinde verimli çalışabilmesi için optimize edilmiştir:

1. **Bellek Yönetimi:**
    a. Dinamik hafıza tahsisi ile gereksiz bellek kullanımı önlenmiştir.
    b. Bağlantılar ve düğümler yalnızca ihtiyaç duyulduğunda oluşturulmuştur.
2. **Zaman Karmaşıklığı Optimizasyonu:**
    a. Algoritmaların zaman karmaşıklıkları detaylıca analiz edilmiş ve iyileştirilmiştir.
    b. Gereksiz döngüler ve hesaplamalar azaltılmıştır.
3. **DEBUG Modu:**
    a. Test ve hata ayıklama süreçlerinde kolaylık sağlamak için DEBUG modu eklenmiştir. Bu mod aktifken veri seti satır satır okunur ve ekrana yazdırılır.
4. **Performans Ölçümü:**
    a. Tüm algoritmaların çalışma süreleri ölçülmüş ve değerlendirilmeye hazır hale getirilmiştir.

7.4 Programın Kullanım Talimatları

Windows ve Linux İçin:

1. **Derleme:** Program, GCC derleyicisi ile aşağıdaki komut kullanılarak derlenebilir:

```
gcc bipartite_graph.c -o bipartite_graph
```

2. **Çalıştırma:** Programı çalıştırmak için aşağıdaki komut kullanılır:

```
./bipartite_graph
```

3. **Veri Seti Kontrolü:** `u.data` dosyasının programla aynı klasörde bulunduğundan emin olun. Aksi halde program çalışmayacaktır.

7.5 Kullanıcıya Sağlanan Kolaylıklar

- **Menü Sistemi:** Kullanıcı, program içinde istediği algoritmayı seçebilir, ID değiştirebilir, performans ve model başarısını ölçebilir.
- **Esneklik:** Program kullanıcıdan gelen yeni ID'lerle işlem yapmaya devam eder.
- **Bilgilendirici Yapı:** Kullanıcı, algoritmaların detaylarını öğrenebilir.

8. Zorluklar ve Çözüm Yolları

Bu proje boyunca karşılaşılan zorluklar ve bu zorlukların üstesinden gelmek için geliştirilen çözümler, projenin başarısında kritik bir rol oynamıştır. İşte proje sürecinde yaşanan temel zorluklar ve bunlara getirilen çözümler:

8.1 Veri Setinin Okunması ve Doğru Şekilde Yorumlanması

Zorluk: MovieLens-100K veri setinin doğru bir şekilde okunması, hatalı veya eksik verilerin tespit edilmesi ve bu verilerin projede kullanılabilir bir formata dönüştürülmesi başlangıçta karmaşık bir süreçti.

Çözüm:

- Veri seti, `u.data` formatında düzenli bir yapıya sahip olduğu için standart dosya okuma yöntemleri kullanılarak okundu.
- Hatalı kullanıcı ve film ID'lerini filtrelemek için koşullar belirlendi (örneğin, kullanıcı ID'sinin 1-943 aralığında olması gerektiği kontrol edildi).
- Her veri satırı işlenirken DEBUG moduyla ara çıktılar alınarak doğrulama yapıldı.
- Timestamp bilgilerinin atlanması gerekiyordu kodu yazarken ilk başta bunu yapmayı unuttuğumuz için hatalı bir grafik oluşturuyorduk.

8.2 Bipartite Graph Yapısının Oluşturulması

Zorluk: Bipartite graph yapısının tasarlanması, düğümler ve kenarlar arasındaki ilişkilerin doğru bir şekilde modellenmesi ve veri yapısının ölçeklenebilir olması gerekiyordu.

Çözüm:

- Kullanıcılar ve filmler ayrı düğüm kümeleri olarak tanımlandı.
- Node ve Edge yapıları dinamik hafıza yönetimiyle oluşturuldu.
- Veri yapısının büyük veri setlerini işlemesini sağlamak için bağlantılar bağlı liste yapısıyla temsil edildi.

8.3 Algoritmaların Tasarımı ve Test Edilmesi

Zorluk: Birden fazla algoritmanın hem doğru bir şekilde çalışmasını sağlamak hem de performansını optimize etmek oldukça zordu. Bazı algoritmalar, özellikle rastgele yürüyüş ve ağırlıklı mesafe tabanlı algoritmalar, hata ayıklama sırasında beklenmedik sonuçlar üretebiliyordu.

Çözüm:

- Her algoritma için ayrıntılı test senaryoları oluşturuldu.
- Algoritmalar, belirli kullanıcı ID'leri üzerinde farklı durumlarda çalıştırılarak hatalar giderildi.
- Algoritma sonuçları veri setindeki gerçek verilere dayanarak doğrulandı.

8.4 Performans Analizi

Zorluk: Algoritmaların zaman ve mekan karmaşıklığını ölçmek ve optimize etmek için kullanılan yöntemlerin tutarlı olması gerekiyordu.

Çözüm:

- Her algoritma için zaman ölçümleri kod içine entegre edildi.
- Karmaşıklık analizleri, algoritmaların çalışma mantığı detaylı bir şekilde incelenerek belirlendi.
- Gereksiz döngüler ve fazlalık işlemler kaldırılarak algoritmalar optimize edildi.

8.5 Model Başarısının Ölçülmesi

Zorluk: Algoritmaların öneri başarısını ölçmek için uygun bir yöntem belirlemek gerekiyordu. Kullanıcılara daha önce izlemedikleri filmler önerildiği için bu filmleri beğenip beğenmeyeceklerini hesaplamamız gerekiyordu. Ve bu süreç epey karmaşıktı.

Çözüm:

- Ortalama puanlara dayalı bir başarı ölçüm sistemi tasarlandı.
- **Precision**, **Recall** ve **F1-Score** metrikleri kullanılarak her algoritmanın öneri başarısı değerlendirildi.
- Rastgele algoritmalar için farklı çalıştırma senaryoları kullanılarak istatistiksel analiz yapıldı.

8.6 Kullanıcı Dostu Arayüz Tasarımı

Zorluk: Menü sisteminin kullanıcı dostu ve sezgisel olması, aynı zamanda tüm işlevleri içermesi gerekiyordu.

Çözüm:

- Program başlatıldığında kullanıcıların hangi işlemleri yapabileceklerini kolayca görebilmesi için detaylı bir menü tasarlandı.
- Kullanıcıdan girilen ID ve menü seçeneklerinin geçerliliği kontrol edildi.
- "Yardım" seçeneği eklenerek her algoritmanın açıklaması programa dahil edildi.

8.7 Türkçe Karakter Sorunları

Zorluk: Türkçe karakterlerin bazı çıktılarda bozuk görüntülenmesi sorunlara yol açtı.

Çözüm:

- `printf` ve diğer çıktı fonksiyonlarında Türkçe karakter kullanılmamasına özen gösterildi.
- Metinler, uyumluluk sorunlarını önlemek için standart İngilizce karakterlerle yeniden yazıldı.

8.8 Zaman Yönetimi

Zorluk: Kapsamlı bir proje olduğundan hem kodlama hem de rapor hazırlama için yeterli zaman ayırmak zordu.

Çözüm:

- Proje aşamaları detaylı bir plana bölündü.
- Her aşama tamamlandıktan sonra test edilip doğrulandı.
- Rapor yazım süreci, kodlama tamamlandıktan sonra başlamadan önce taslaklarla desteklendi.

9. Sonuç ve Gelecek Çalışmalar

Bu proje, **Bipartite Graph** yapısını kullanarak film öneri sistemlerinin nasıl çalıştığını derinlemesine anlamayı ve uygulamayı hedeflemiştir. Farklı algoritmaların performanslarını değerlendirme ve modelleme sürecindeki detaylı çalışmalar, proje boyunca önemli sonuçlar elde edilmesini sağlamıştır. Bu bölümde, projeden çıkarılan sonuçlar, mevcut sistemin iyileştirilebileceği alanlar ve gelecekte uygulanabilecek daha gelişmiş yaklaşımlar ele alınmıştır.

9.1 Projeden Çıkarılan Sonuçlar

1. **Bipartite Graph ile Modelleme:**
    a. Kullanıcılar ve filmler arasındaki ilişkileri modellemek için **Bipartite Graph** yapısı uygun ve etkili bir yöntemdir.
    b. Bu yapı sayesinde, farklı algoritmaların aynı veri yapısı üzerinde kolayca uygulanabildiği görülmüştür.
2. **Algoritmaların Çeşitliliği ve Performansı:**
    a. Basit ve hızlı algoritmalar (örneğin, Rastgele Film Önerisi) genellikle daha düşük başarı gösterirken, daha karmaşık algoritmalar (örneğin, Dinamik Kullanıcıya Özel Film Önerisi) daha yüksek başarı oranlarına ulaşmıştır.
    b. Dinamik ve kullanıcıya özgü algoritmaların, film önerisi sistemlerinde kişiselleştirme açısından önemli olduğu sonucuna varılmıştır.
3. **Zaman ve Mekan Karmaşıklığı:**
    a. Algoritmaların zaman ve mekan karmaşıklıkları detaylı şekilde analiz edilmiştir.
    b. Rastgele ve basit algoritmaların düşük zaman karmaşıklığına sahip olduğu, buna karşın dinamik veya mesafe tabanlı algoritmaların daha yüksek zaman karmaşıklığına neden olduğu görülmüştür.
4. **Model Başarısı:**
    a. **Precision, Recall ve F1-Score** gibi metrikler kullanılarak algoritmaların öneri başarısı değerlendirilmiştir.
    b. Rastgele algoritmaların başarısız olduğu ancak dinamik ve yüksek dereceli algoritmaların tutarlı ve yüksek başarı elde ettiği görülmüştür.
5. **Kullanıcı Deneyimi:**
    a. Kullanıcı dostu bir menü sistemi, algoritmaların test edilmesini kolaylaştırmış ve programın genel kullanılabilirliğini artırmıştır.

9.2 Mevcut Sistemin İyileştirilebileceği Alanlar

1. **Veri Seti ve Ölçeklenebilirlik:**
    a. MovieLens-100K veri seti, küçük ölçekli bir veri seti olarak kullanılmıştır. Daha büyük veri setleriyle (örneğin, MovieLens-1M) sistemin performansı ve ölçeklenebilirliği değerlendirilebilir.
2. **Optimizasyon:**
    a. Mesafe tabanlı algoritmaların zaman karmaşıklığı yüksektir. Bu algoritmalar için daha hızlı yollar veya veri yapıları (örneğin, Fibonacci heap) kullanılabilir.
    b. Rastgele yürüyüş algoritmasının doğruluk oranını artırmak için ek parametreler veya daha gelişmiş stratejiler uygulanabilir.
3. **Kullanıcı Etkileşimi:**
    a. Menü sistemi daha etkileşimli hale getirilebilir.
    b. Kullanıcıya, önerilen filmleri direkt olarak kaydetme veya favorilere ekleme seçenekleri sunulabilir.

9.3 Gelecekte Uygulanabilecek Daha Gelişmiş Algoritmalar

1. **Collaborative Filtering (İşbirlikçi Filtreleme):**
    a. Kullanıcıların benzer tercihlerini analiz ederek öneriler yapmak, sistemin kişiselleştirme başarısını artırabilir.
2. **Content-Based Filtering (İçerik Tabanlı Filtreleme):**
    a. Filmlerin türü, oyuncuları veya yönetmeni gibi içerik bilgilerine dayalı öneriler eklenebilir.
3. **Hybrid Systems (Hibrit Sistemler):**
    a. İşbirlikçi ve içerik tabanlı filtreleme algoritmalarını birleştirerek daha güçlü öneri sistemleri tasarlanabilir.
4. **Makine Öğrenimi ve Derin Öğrenme:**
    a. Kullanıcı ve film özelliklerini modellemek için sinir ağları ve derin öğrenme yöntemleri uygulanabilir.
    b. Önerilerin doğruluğunu artırmak için geçmiş kullanıcı etkileşimlerinden öğrenen algoritmalar geliştirilebilir.
5. **Gerçek Zamanlı Öneriler:**
    a. Kullanıcının gerçek zamanlı davranışlarına göre öneriler üretebilen algoritmalar (örneğin, kullanıcı bir filmi ararken benzer filmleri önerme) sisteme entegre edilebilir.
6. **Daha Gelişmiş Random Walk Algoritmaları:**
    a. Rastgele yürüyüş algoritmaları, kullanıcıların izleme alışkanlıklarına göre uyarlanabilir.
    b. Yeniden başlama olasılıkları veya ağırlıklandırmalar daha etkili öneriler için optimize edilebilir.

9.4 Genel Değerlendirme

Bu proje, film öneri sistemlerinin temel yapı taşlarını anlamak ve uygulamak için güçlü bir temel oluşturmuştur. Elde edilen sonuçlar, mevcut sistemin başarılarını ve sınırlamalarını ortaya koyarken, gelecekteki geliştirmeler için de değerli bir yol haritası sunmaktadır.

Sonuç olarak, öneri sistemleri, hem veri yapısı hem de algoritma tasarımı açısından sürekli gelişim gösteren bir alandır. Bu projede yapılan çalışmalar, hem akademik hem de endüstriyel uygulamalar için önemli bir referans niteliği taşımaktadır.